

Core PHP + SQL — CRUD Quiz Workbook

Understand it, practice it on paper, and predict what sir asks.

update • delete • view • edit • login • register — by id / email / reg-no / URL

Paper quiz. Classic core PHP style: HTML form -> \$_POST, link -> \$_GET.

1. The ONE skeleton to memorise

Every answer is built from this. Learn to write these lines cold. Only three ideas: **connect -> build SQL -> run it.**

```
<?php
$conn = mysqli_connect("localhost", "root", "", "dbname"); // 1. connect

$id = $_GET['id']; // input from the URL (?id=3)
$name = $_POST['name']; // input from an HTML form

$sql = "...your SQL here..."; // 2. build the query
$result = mysqli_query($conn, $sql); // 3. run it
?>
```

connection.php (usually kept separate and included):

```
$conn = mysqli_connect("localhost", "root", "", "yaqeen_web_project");
include("connection.php"); // other files pull it in
```

XAMPP defaults: host = localhost, user = root, password = empty. Last argument = your database name.

2. The five operations (write each cold)

CREATE -> INSERT (register / add)

```
$name = $_POST['name']; $email = $_POST['email'];
$sql = "INSERT INTO users (full_name, email) VALUES ('$name', '$email')";
mysqli_query($conn, $sql);
```

READ -> SELECT (view / list all)

```
$result = mysqli_query($conn, "SELECT * FROM users");
while ($row = mysqli_fetch_assoc($result)) { // loop every row
    echo $row['full_name'] . " - " . $row['email'] . "<br>";
}
```

UPDATE -> UPDATE ... WHERE (update / edit)

```
$id = $_POST['id']; $name = $_POST['name'];
$sql = "UPDATE users SET full_name='$name' WHERE id='$id'";
mysqli_query($conn, $sql);
```

DELETE -> DELETE ... WHERE

```
$id = $_GET['id']; // delete link: delete.php?id=3
$sql = "DELETE FROM users WHERE id='$id'";
mysqli_query($conn, $sql);
```

LOGIN -> SELECT + count

```
$email = $_POST['email']; $pass = $_POST['password'];
$result = mysqli_query($conn, "SELECT * FROM users WHERE email='$email' AND user_password='$pass'");
if (mysqli_num_rows($result) == 1) { echo "Login successful"; }
else { echo "Invalid email or password"; }
```

mysqli_num_rows == 1 -> exactly one matching row -> the login is correct.

3. Sir's twist: change the key or the input source

Same query - only the **WHERE** column and where the value **comes from** change. This is where the marks are.

Change the key (the WHERE column)

```
... WHERE id      = '$id'           -- by id
... WHERE email   = '$email'        -- by email
... WHERE reg_no  = '$regno'        -- by reg number
... WHERE id='$id' AND email='$email' -- by id AND email (sir's note)
```

Change the URL -> read with \$_GET instead of \$_POST

```
$id    = $_GET['id'];           // URL:  update.php?id=3
$email = $_GET['email'];        // URL:  delete.php?email=abc@x.com
```

Rule of thumb: delete usually uses **\$_GET** (a link with ?id=). Register / update usually use **\$_POST** (a form). If sir says "do it from the URL", switch that line to **\$_GET**.

4. Practice - write each on paper, then check

Cover the answers. Attempt from memory first. Lines are provided to write on.

Q1. Delete a product where id comes from the URL.

Q2. Update a student's email based on reg_no (form data).

Q3. View all products and echo each title.

Q4. Register a user (name, email, password) into users.

Q5. Write the login check.

Q6. Delete a user by email taken from the URL.

Q7. Update a user's status to 'Active' based on id AND email.

Q8. View ONE user whose id comes from the URL.

5. Answer key

Q1. Delete a product where id comes from the URL.

```
$id = $_GET['id'];  
mysqli_query($conn, "DELETE FROM products WHERE id='$id'");
```

Q2. Update a student's email based on reg_no (form data).

```
$regno = $_POST['regno']; $email = $_POST['email'];  
mysqli_query($conn, "UPDATE students SET email='$email' WHERE reg_no='$regno'");
```

Q3. View all products and echo each title.

```
$r = mysqli_query($conn, "SELECT * FROM products");  
while ($row = mysqli_fetch_assoc($r)) { echo $row['title'] . "<br>"; }
```

Q4. Register a user (name, email, password) into users.

```
$n=$_POST['name']; $e=$_POST['email']; $p=$_POST['password'];  
mysqli_query($conn, "INSERT INTO users (full_name,email,user_password) VALUES ('$n','$e','$p')");
```

Q5. Write the login check.

```
$e=$_POST['email']; $p=$_POST['password'];  
$r=mysqli_query($conn,"SELECT * FROM users WHERE email='$e' AND user_password='$p'");  
if (mysqli_num_rows($r)==1){ echo "Login successful"; } else { echo "Invalid"; }
```

Q6. Delete a user by email taken from the URL.

```
$email = $_GET['email'];  
mysqli_query($conn, "DELETE FROM users WHERE email='$email'");
```

Q7. Update a user's status to 'Active' based on id AND email.

```
$id=$_POST['id']; $email=$_POST['email'];  
mysqli_query($conn, "UPDATE users SET status='Active' WHERE id='$id' AND email='$email'");
```

Q8. View ONE user whose id comes from the URL.

```
$id = $_GET['id'];  
$r = mysqli_query($conn, "SELECT * FROM users WHERE id='$id'");  
$row = mysqli_fetch_assoc($r);  
echo $row['full_name'];
```

6. Top probables - what sir most likely asks

Likelihood	Sir asks	Where to look
~90%	Write PHP to UPDATE a record (by id / email)	Sec 2 UPDATE + Sec 3
~90%	Write PHP to DELETE a record (by id, from URL)	Sec 2 DELETE (\$_GET)
~80%	Write PHP to VIEW / display all records	Sec 2 READ (while + fetch_assoc)
~75%	How does login / register STORE to DB? Write it.	Sec 2 CREATE + LOGIN
~70%	TWIST: now do it by email / reg_no / from URL	Sec 3
~50%	Just write the SQL query (update/delete/view)	Sec 7
~40%	What does mysqli_query / num_rows / fetch_assoc do?	Sec 8
~30%	Explain React -> Backend -> refresh flow	Sec 9

7. The SQL alone (if he asks queries only)

```

INSERT INTO users (full_name, email) VALUES ('Ali', 'ali@x.com');
SELECT * FROM users;
SELECT * FROM users WHERE id = 3;
UPDATE users SET email = 'new@x.com' WHERE id = 3;
DELETE FROM users WHERE id = 3;

```

8. The mysqli functions you must name

Function	What it does
mysqli_connect(host,user,pass,db)	Opens the connection to MySQL.
mysqli_query(\$conn, \$sql)	Runs ANY SQL query.
mysqli_fetch_assoc(\$result)	Gets one row as an array; loop it for many rows.
mysqli_num_rows(\$result)	Counts matched rows (login uses == 1).
mysqli_error(\$conn)	Returns the last database error text.
\$_POST['x'] / \$_GET['x']	Reads form data / URL data.

9. Sir's React diagram (in case he asks)

In React you only write a **submit()** that SENDS data (it writes, it does not fetch). The updated table shows on the next load, when the SELECT (view) runs again.

```

React  --update-->  Backend (PHP runs UPDATE/DELETE/INSERT)
  |                  |
  v Table Show      | refresh
Table <-----+

```

Say it as: **submit = write only; view = a separate SELECT on refresh.**

CRUD = Create(INSERT) - Read(SELECT) - Update(UPDATE) - Delete(DELETE). Memorise the skeleton, change the WHERE, pick \$_POST or \$_GET. That is the whole quiz.